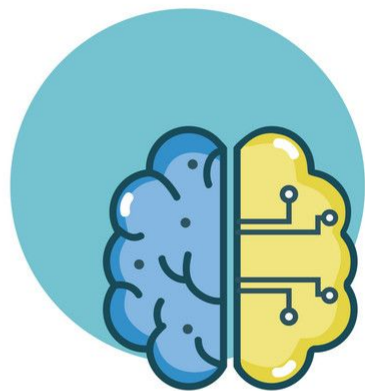


INTRODUCTION TO MACHINE LEARNING

GRADIENT DESCENT

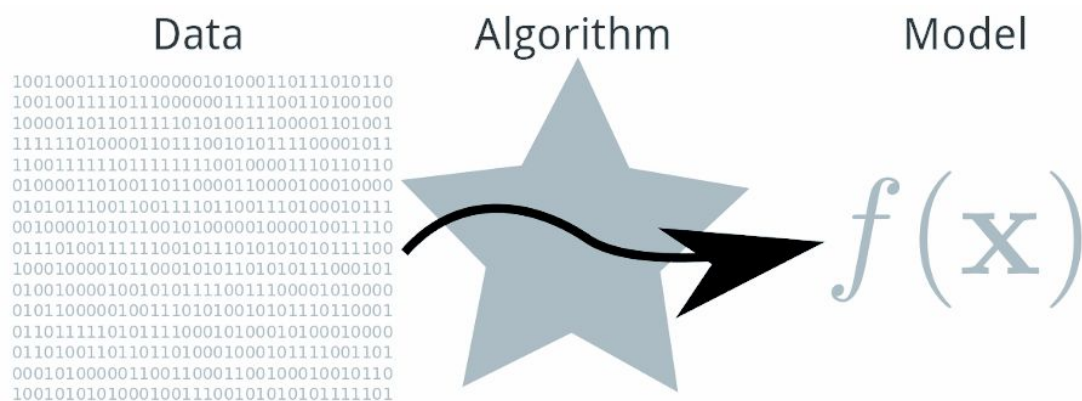


Elisa Ricci



MACHINE LEARNING IDEA

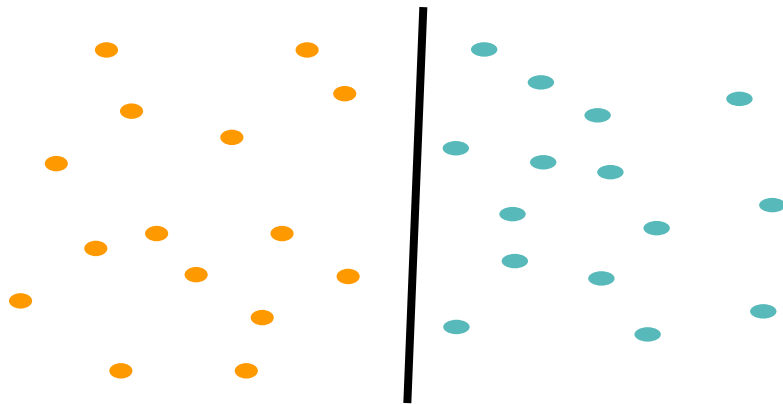
- ML allows computers to acquire knowledge.
- Knowledge is acquired through **algorithms** by learning and inferring from **data**.
- Knowledge is represented by a **model**.
- The model is used on future data.



LINEAR MODELS

A **linear model** is a model assumes that the data are linearly separable

Assume a specific hypothesis space, i.e. linear functions



LINEAR MODELS

A linear model in n -dimensional space (i.e. n features) is defined by $n+1$ weights.

In two dimensions, we have a line:

$$0 = w_1 f_1 + w_2 f_2 + b$$

In three dimensions, a plane:

$$0 = w_1 f_1 + w_2 f_2 + w_3 f_3 + b$$

In n -dimensions, a **hyperplane**

$$0 = b + \sum_{i=1}^n w_i f_i$$

PERCEPTRON LEARNING ALGORITHM

repeat until convergence (or for some # of iterations):

for each training example $(f_1, f_2, \dots, f_n, \text{label})$:

$$\text{prediction} = b + \sum_{i=1}^n w_i f_i$$

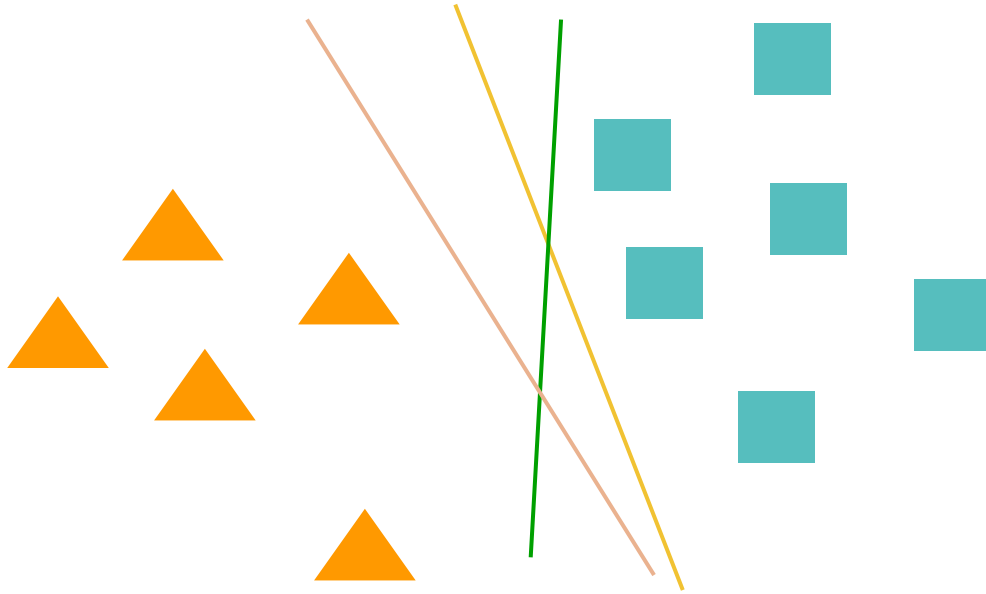
if $\text{prediction} * \text{label} \leq 0$: // they don't agree

for each w_i :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

WHICH LINE WILL THE PERCEPTRON FIND?



Only guaranteed to find **some** line that separates the data!

LINEAR MODELS

Perceptron algorithm is one example of a linear classifier

Many, many other algorithms learn a line (i.e. a setting of a linear combination of weights)

Goals:

- Explore a number of linear training algorithms
- Understand why these algorithms work

PERCEPTRON LEARNING ALGORITHM

repeat until convergence (or for some # of iterations):

for each training example $(f_1, f_2, \dots, f_n, \text{label})$:

$$\text{prediction} = b + \sum_{i=1}^n w_i f_i$$

if $\text{prediction} * \text{label} \leq 0$: // they don't agree

for each w_i :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

A CLOSER LOOK AT WHY WE GOT IT WRONG

$$\overset{w_1}{1} * \overset{w_2}{f_1} + 0 * f_2 =$$

$$1 * -1 + 0 * 1 = -1$$

$$(-1, 1) \quad +$$

← This value should be positive!

↑
contributed in the
wrong direction

←
could have contributed
(positive feature) but it did
not since the weight is 0

A CLOSER LOOK AT WHY WE GOT IT WRONG

$$\overset{w_1}{1} * \overset{w_2}{f_1} + 0 * f_2 =$$

$$1 * -1 + 0 * 1 = -1$$

$(-1, 1)$ $+$

← This value should be positive!

↑
decrease
e.g. from 1 to 0

←
increase
from 0 to 1

MODEL-BASED MACHINE LEARNING

1. Pick a model

e.g. a hyperplane, a decision tree,...

A model is defined by a collection of parameters

What are the parameters for DT? Perceptron?

MODEL-BASED MACHINE LEARNING

1. Pick a model

e.g. a hyperplane, a decision tree,...

A model is defined by a collection of parameters

DT: the structure of the tree, which features each node splits on, the predictions at the leaves

Perceptron: the weights and the bias value

MODEL-BASED MACHINE LEARNING

1. Pick a model
e.g. a hyperplane, a decision tree,...
2. Pick a criterion to optimize (aka objective function)

What criteria do DT learning and perceptron learning optimize?

MODEL-BASED MACHINE LEARNING

1. Pick a **model**
 - e.g. a hyperplane, a decision tree,...
2. Pick a criterion to optimize (aka **objective function**)
 - e.g. training error
3. Develop a **learning algorithm**
 - the algorithm should try and minimize the criteria, sometimes in a heuristic way (i.e. non-optimally), sometimes exactly

LINEAR MODELS IN GENERAL

1. Pick a model

$$0 = b + \sum_{j=1}^m w_j f_j$$


These are the parameters we want to learn

2. Pick a criterion to optimize (aka objective function)

SOME NOTATION: INDICATOR FUNCTION

Convenient notation for turning *True* and *False* answers into numbers/counts:

$$1[x] = \begin{cases} 1 & \text{if } x = \text{True} \\ 0 & \text{if } x = \text{False} \end{cases}$$

SOME NOTATION: DOT-PRODUCT

We use a **vector notation**

We represent an example $f_1, f_2, \dots, f_m$ as a single vector, $\mathbf{x}$

- j subscript will indicate feature indexing, i.e., x_j
- i subscript will indicate examples indexing over a dataset, i.e., $\mathbf{x}_i$

Similarly, we can represent the weight $w_1, w_2, \dots, w_m$ as a single vector, $\mathbf{w}$

The dot-product between two vectors a and b is defined as:

$$a \cdot b = \sum_{j=1}^m a_j b_j$$

LINEAR MODELS

1. Pick a model
2. Pick a criterion to optimize (aka objective function)

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

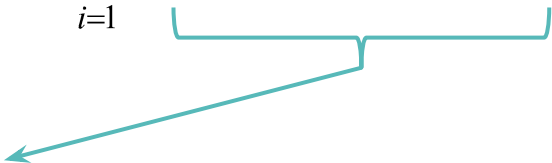
What does this equation say?

0/1 LOSS FUNCTION

$$\sum_{i=1}^n 1[y_i(\underbrace{w \cdot x_i + b}_{\text{distance from hyperplane}}) \leq 0]$$

- distance from hyperplane
- sign is prediction

0/1 LOSS FUNCTION

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$


whether or not the
prediction and label agree,
true if *they don't*

total number of
mistakes, aka 0/1 loss

MODEL-BASED MACHINE LEARNING

1. Pick a model

$$0 = b + \sum_{j=1}^m w_j f_j$$

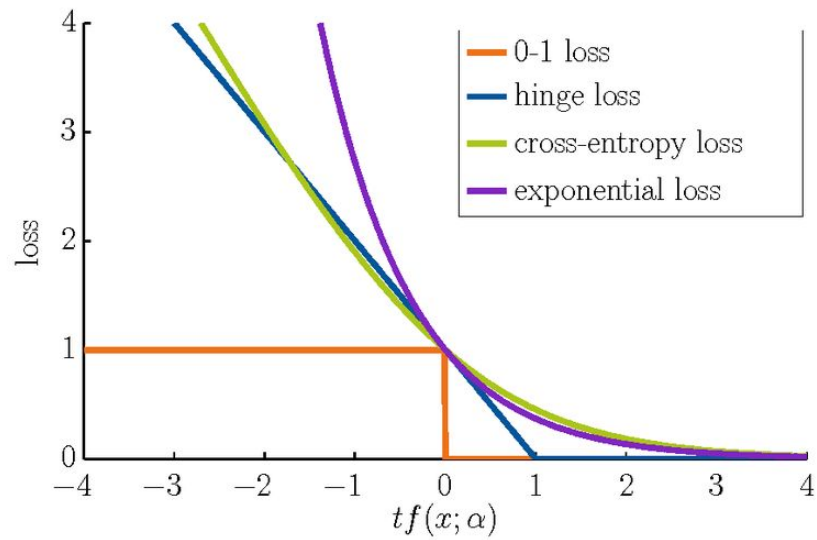
2. Pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

3. Develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

Find w and b that
minimize the 0/1 loss
(i.e. training error)



LOSS FUNCTIONS

MINIMIZING 0/1 LOSS

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

Find w and b that
minimize the 0/1 loss
(i.e. training error)

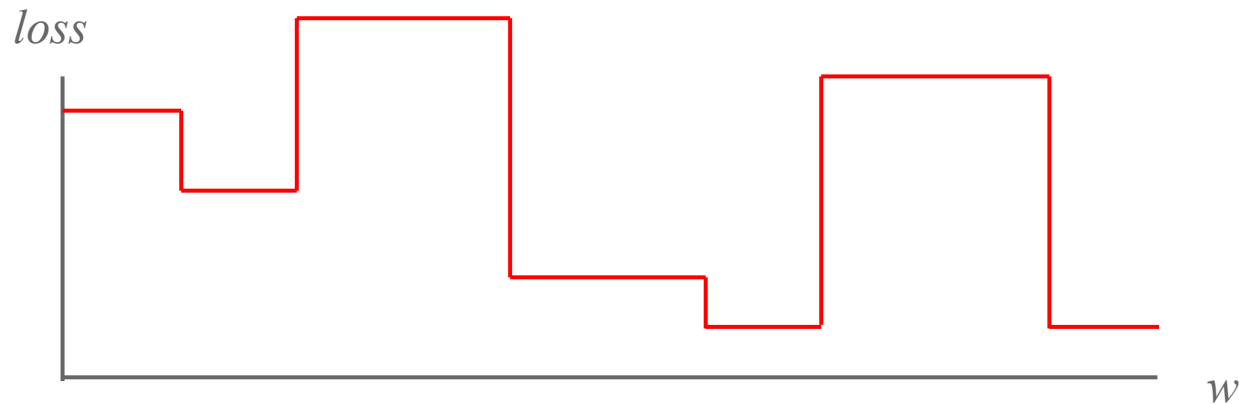
How do we do this?

How do we *minimize* a function?

Why is it hard for this function?

MINIMIZING 0/1 IN ONE DIMENSION

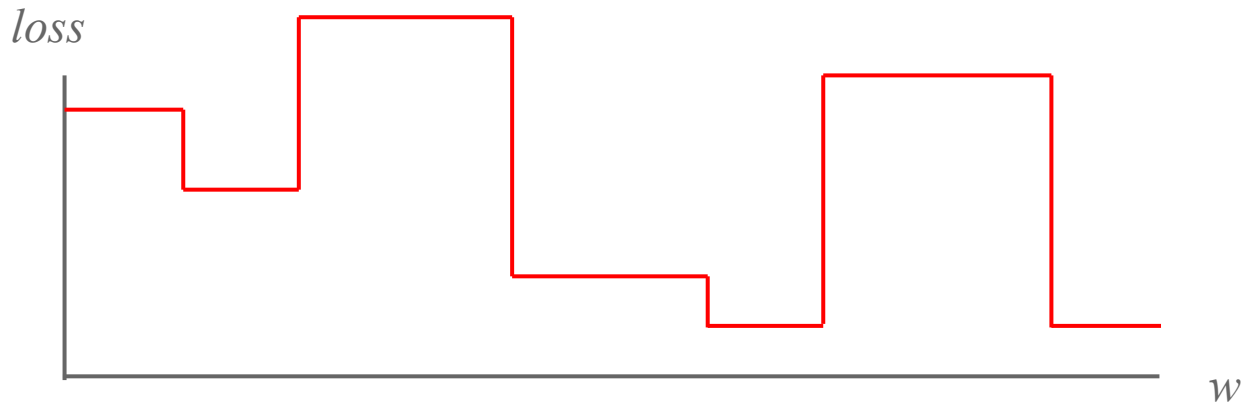
$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$



Each time we change *w* such that the example is right/wrong the loss will increase/decrease

MINIMIZING 0/1 IN ONE DIMENSION

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$



Each new feature we add (i.e. weights) adds another dimension to this space!

MINIMIZING 0/1 LOSS

Find w and b that
minimize the 0/1 loss

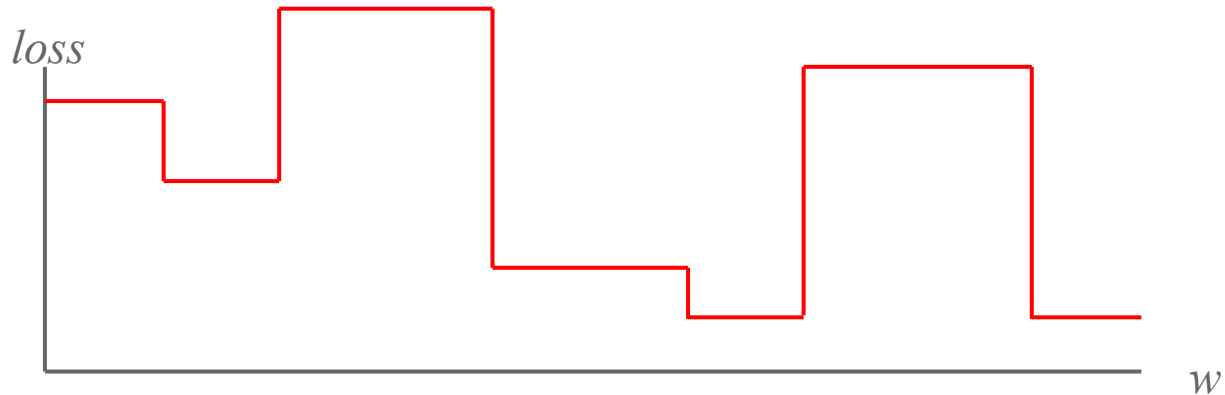
$$\operatorname{argmin}_{w,b} \sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

This turns out to be hard (in fact, NP-HARD 😞)

Challenge:

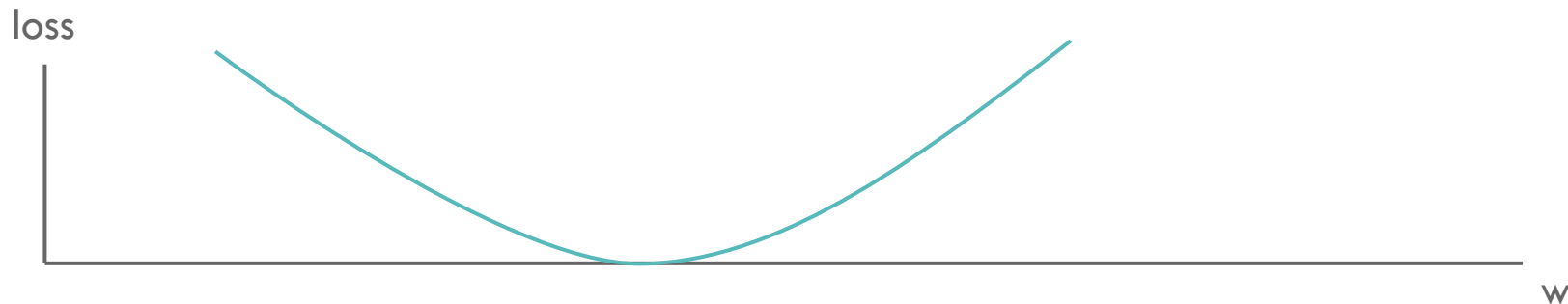
- Small changes in any w can have large changes in the loss (the change isn't continuous)
- There can be many, many local minima
- At any given point, we don't have much information to direct us towards any minima

MORE MANAGEABLE LOSS FUNCTIONS



What properties do we want from our loss function?

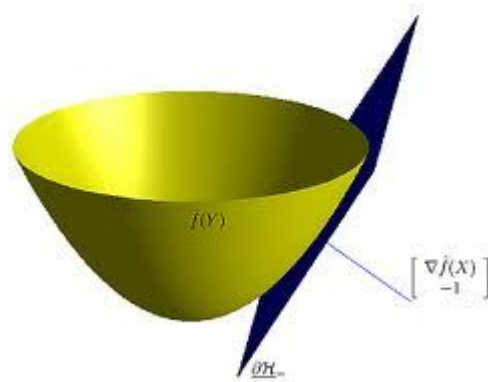
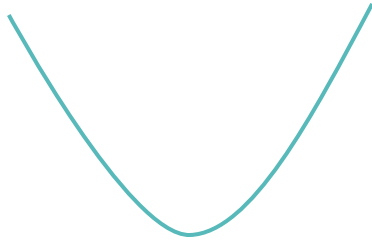
MORE MANAGEABLE LOSS FUNCTIONS



- Ideally, continuous (i.e. differentiable) so we get an indication of direction of minimization
- Only one minima

CONVEX FUNCTIONS

Convex functions look something like:



One definition: The line segment between any two points on the function is *above* the function

SURROGATE LOSS FUNCTIONS

For many applications, we really would like to minimize the 0/1 loss

A **surrogate loss function** is a loss function that provides an upper bound on the actual loss function (in this case, 0/1)

We would like to identify convex surrogate loss functions to make them easier to minimize

Key to a loss function: how it scores the difference between the actual label y and the predicted label y'

SURROGATE LOSS FUNCTIONS

0/1 loss:

$$l(y, y') = 1[y y' \leq 0]$$

Ideas?

Some function that is a proxy for error, but is continuous and convex

SURROGATE LOSS FUNCTIONS

0/1 loss: $l(y, y') = 1[y y' \leq 0]$

Hinge: $l(y, y') = \max(0, 1 - y y')$

Exponential: $l(y, y') = \exp(-y y')$

Squared loss: $l(y, y') = (y - y')^2$

Why do these work? What do they penalize?

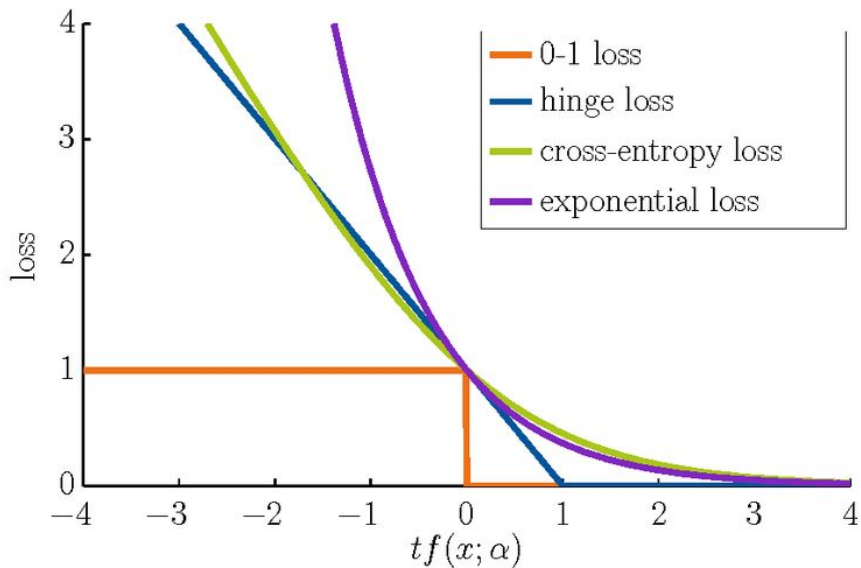
SURROGATE LOSS FUNCTIONS

0/1 loss: $l(y, y') = 1[y y' \leq 0]$

Hinge: $l(y, y') = \max(0, 1 - y y')$

Squared loss: $l(y, y') = (y - y')^2$

Exponential: $l(y, y') = \exp(-y y')$



MODEL-BASED MACHINE LEARNING

1. pick a model

$$0 = b + \sum_{j=1}^m w_j f_j$$

2. pick a criteria to optimize (aka objective function)

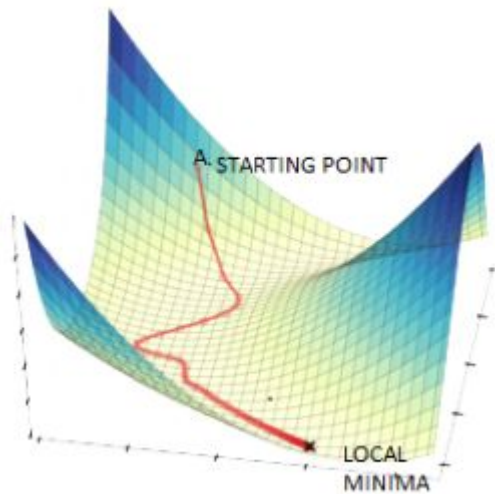
$$\sum_{i=1}^n \exp(-y_i(w \cdot x_i + b))$$

use a convex surrogate
loss function

3. develop a learning algorithm

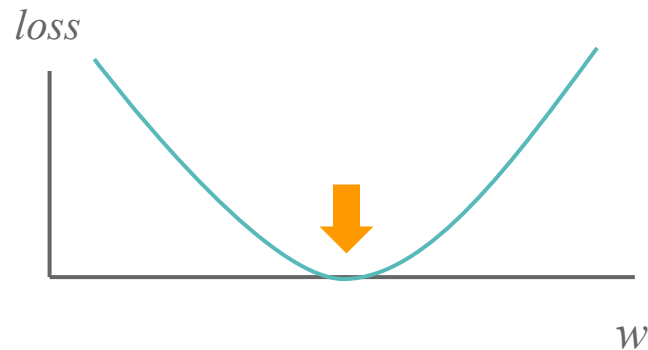
$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b))$$

Find w and b that minimize
the surrogate loss



GRADIENT DESCENT

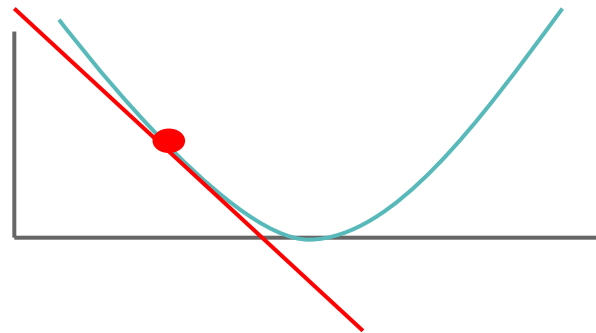
FINDING THE MINIMUM



How do we find the minimum for a function?

ONE APPROACH: GRADIENT DESCENT

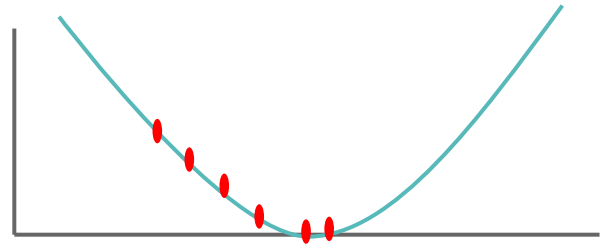
Partial derivatives give us the slope (i.e. direction to move) in that dimension



ONE APPROACH: GRADIENT DESCENT

Approach:

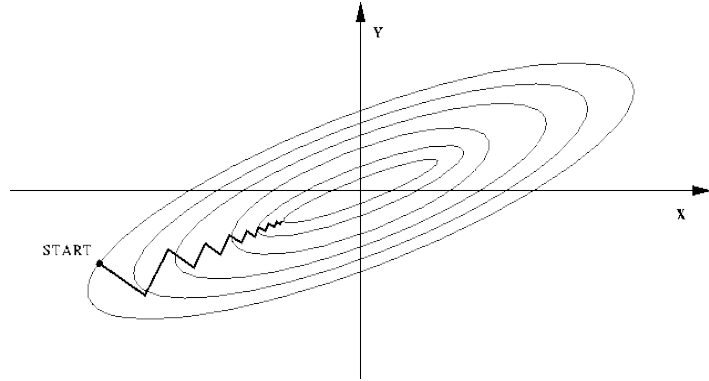
- pick a starting point (w)
- repeat:
 - pick a dimension
 - move a small amount in that dimension towards decreasing loss (using the derivative)



ONE APPROACH: GRADIENT DESCENT

Approach:

- pick a starting point (w)
- repeat:
 - pick a dimension
 - move a small amount in that dimension towards decreasing loss (using the derivative)



GRADIENT DESCENT

- Pick a starting point (w)
- Repeat until loss doesn't decrease in any dimension:
 - pick a dimension
 - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \frac{d}{dw_j} loss$$

Why negative?



GRADIENT DESCENT

$$w_j = w_j - \eta \frac{d}{dw_j} loss$$

Learning
rate

How much we want to move
in the error direction, often
this will change over time

SOME MATH

$$\begin{aligned}\frac{d}{dw_j} loss &= \frac{d}{dw_j} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) \\ &= \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) \frac{d}{dw_j} (-y_i(w \cdot x_i + b))\end{aligned}$$

SOME MATH

$$\begin{aligned}\frac{d}{dw_j} loss &= \frac{d}{dw_j} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) \\ &= \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) \frac{d}{dw_j} - y_i(w \cdot x_i + b) \\ &= \sum_{i=1}^n -y_i x_{ij} \exp(-y_i(w \cdot x_i + b))\end{aligned}$$

GRADIENT DESCENT

For our choice of the loss we have:

$$w_j = w_j - \eta \frac{d}{dw_j} \text{loss}(w)$$

$$w_j = w_j + \eta \sum_{i=1}^n y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

What is this doing?

EXPONENTIAL UPDATE RULE

$$w_j = w_j + \eta \sum_{i=1}^n y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

for each example x_i :

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

Does this look familiar?

PERCEPTRON LEARNING ALGORITHM!

repeat until convergence (or for some # of iterations):

for each training example $(f_1, f_2, \dots, f_n, \text{label})$:

$$\text{prediction} = b + \sum_{i=1}^n w_i f_i$$

if $\text{prediction} * \text{label} \leq 0$: // they don't agree

for each w_i :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$$

In practice $w_j = w_j + x_{ij} y_i c$ where $c = \eta \exp(-y_i(w \cdot x_i + b))$

PERCEPTRON LEARNING ALGORITHM!

repeat until convergence (or for some # of iterations):

for each training example $(f_1, f_2, \dots, f_n, \text{label})$:

$$\text{prediction} = b + \sum_{i=1}^n w_i f_i$$

if $\text{prediction} * \text{label} \leq 0$: // they don't agree

for each w_i :

$$w_i = w_i + f_i * \text{label}$$

Note: for gradient descent, we always update

$$b = b + \text{label}$$

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$$

In practice $w_j = w_j + x_{ij} y_i c$ where $c = \eta \exp(-y_i(w \cdot x_i + b))$

THE CONSTANT

$$c = \eta \exp(-y_i(w \cdot x_i + b))$$

learning rate

label

prediction

When is this large/small?

THE CONSTANT

$$c = \eta \exp(-y_i(w \cdot x_i + b))$$

label

prediction

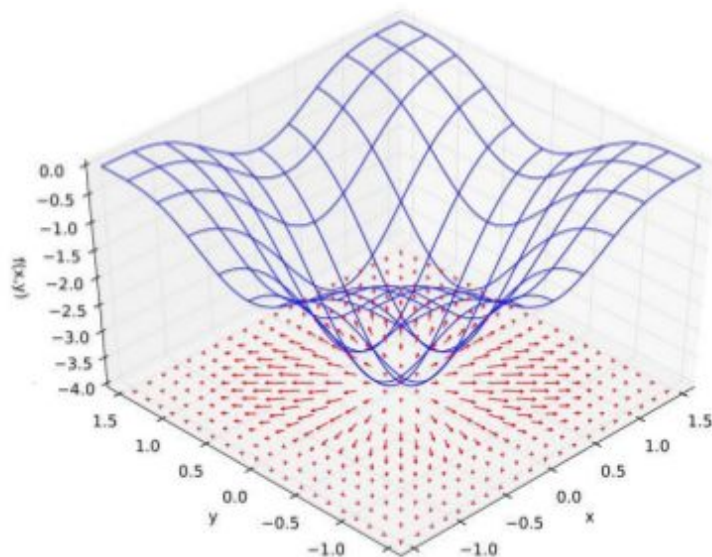
- If they are the same sign, as the predicted gets larger there update gets smaller
- If they are different, the more different they are, the bigger the update

GRADIENT

- The gradient is the vector of partial derivatives wrt to all the coordinates of the weights:

$$\nabla_{\mathbf{w}} L = \left[\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \quad \cdots \quad \frac{\partial L}{\partial w_N} \right]$$

- Each partial derivative measures how fast the loss changes in one direction.
- When the gradient is zero, i.e. all the partials derivatives are zero, the loss is not changing in any direction.
- Note: the arrows (gradients) point out from a minimum toward a maximum.

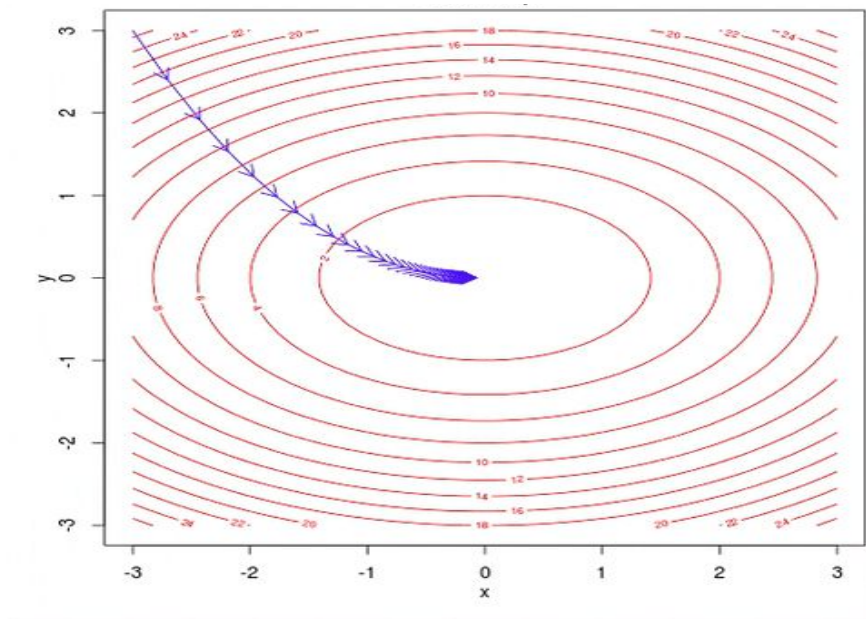


GRADIENT DESCENT

Algorithm 21 GRADIENTDESCENT($\mathcal{F}, K, \eta_1, \dots$)

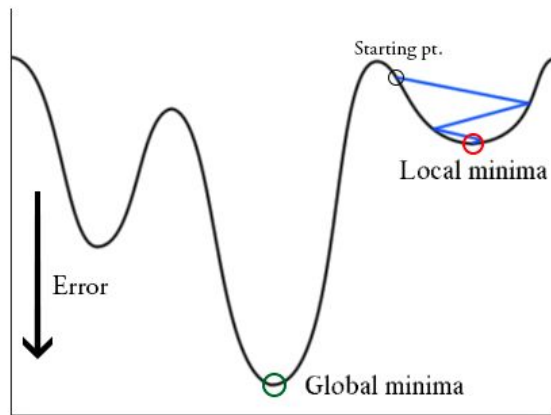
```
1:  $\mathbf{z}^{(0)} \leftarrow \langle o, o, \dots, o \rangle$  // initialize variable we are optimizing
2: for  $k = 1 \dots K$  do
3:    $\mathbf{g}^{(k)} \leftarrow \nabla_{\mathbf{z}} \mathcal{F}|_{\mathbf{z}^{(k-1)}}$  // compute gradient at current location
4:    $\mathbf{z}^{(k)} \leftarrow \mathbf{z}^{(k-1)} - \eta^{(k)} \mathbf{g}^{(k)}$  // take a step down the gradient
5: end for
6: return  $\mathbf{z}^{(K)}$ 
```

GRADIENT DESCENT

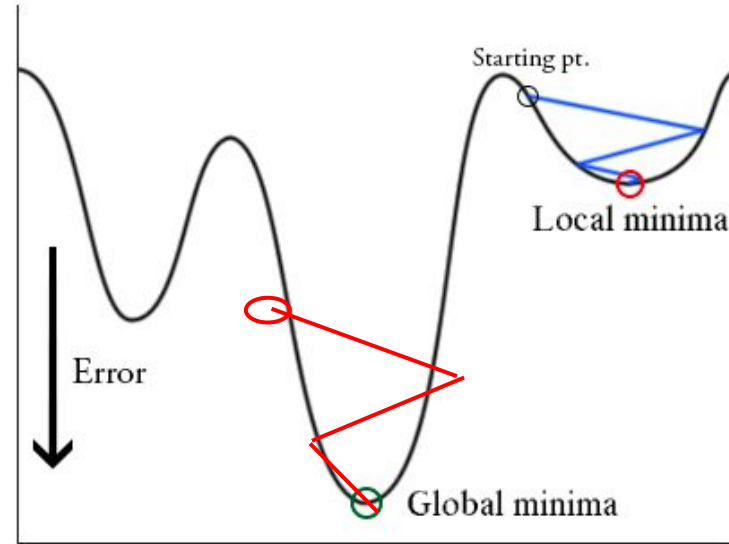


GRADIENT DESCENT

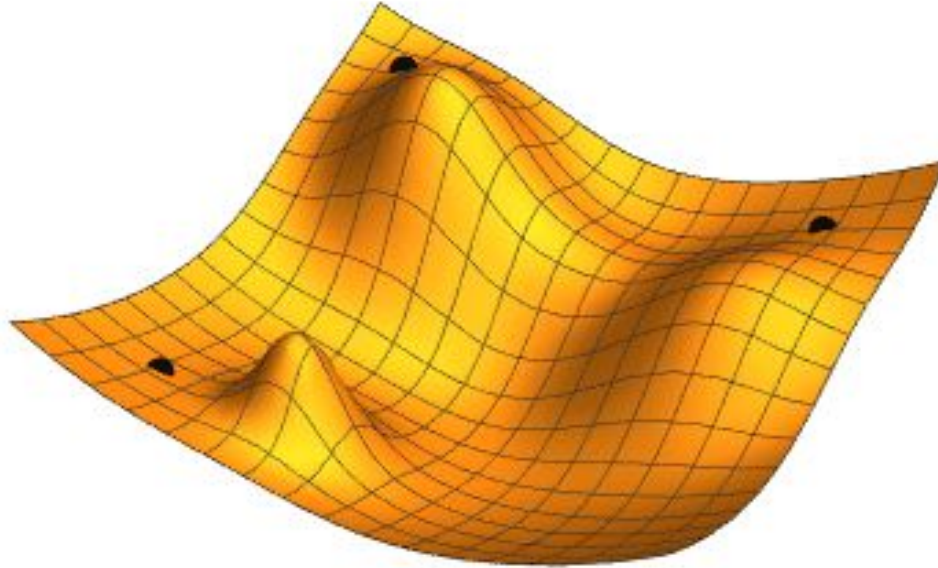
- In problems where the optimization problem is non-convex, it probably has **local minima**.
- An example is neural network. For long time people did not use neural networks, because they preferred methods that guaranteed finding the optimal solution.



GRADIENT DESCENT

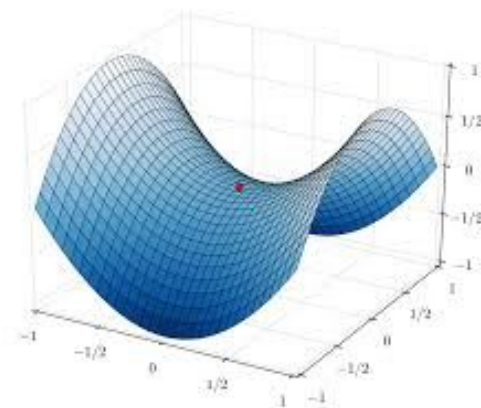


GRADIENT DESCENT



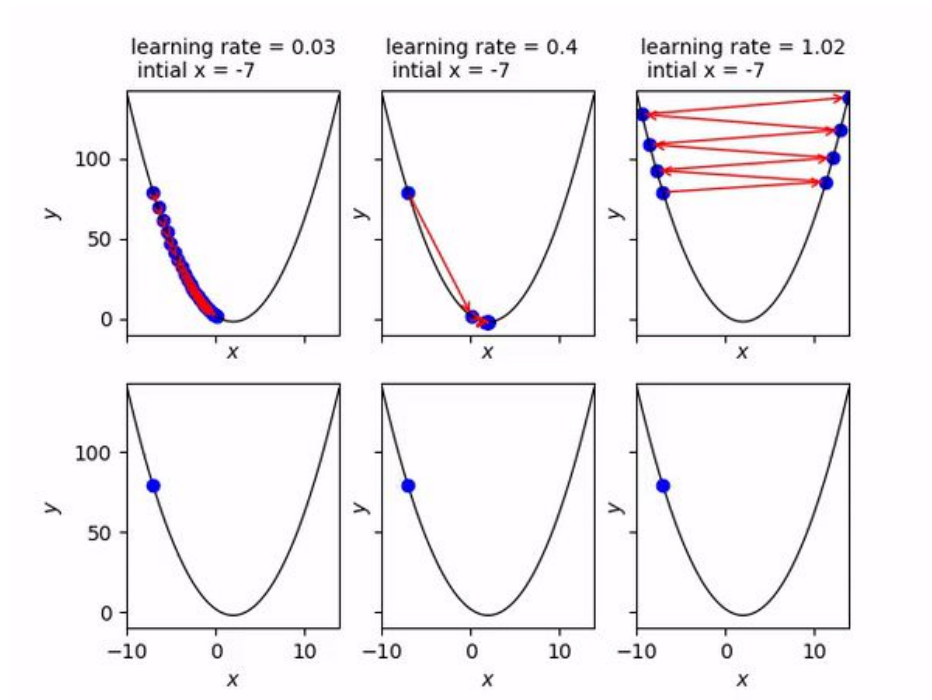
GRADIENT DESCENT

- **Saddle point:** Some directions curve upwards, and others curve downwards.
- At a saddle point, the gradient is 0 even if we are not at a minimum.
 - If we are exactly on the saddle point, then we are stuck.
 - If we are slightly to the side, then we can get unstuck.
- **Saddle points very common in high dimensions!**



LEARNING RATE

Very important hyper-parameter



SUMMARY

- Model-based machine learning:
 - define a model, objective function (i.e. loss function), minimization algorithm
- Gradient descent minimization algorithm
 - so far we consider the case where the loss function is convex
 - make small updates towards lower losses
- Perceptron learning algorithm and gradient descent with exponential loss function are similar (modulo a learning rate)
- Gradient descent in general

QUESTIONS?



Some slides are taken from David Kauchak